Software Requirements Specification

For

CPU Scheduling Algorithm Simulator

Version 1.0

Prepared by Jonathan Koral

Brooklyn College

February 17, 2026

Table of Contents

Table of Contents 1
Revision History 1
1. Introduction 2
1.1 Purpose 2
1.2 Document Conventions 2
1.3 Intended Audience and Reading Suggestions 2
1.4 Product Scope 2
1.5 References 2
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 3
2.5 Design and Implementation Constraints 3
2.6 User Documentation 4
3. External Interface Requirements 4
3.1 User Interfaces 4
3.2 Hardware Interfaces 5
3.3 Software Interfaces 5
3.4 Communications Interfaces 5
4. System Features 5
4.1 Input Processes 5
4.2 Algorithm Selection 6
4.3 Simulation 6
4.4 Gantt Chart Visualization 7
4.5 Process Table and Relevant Data 7
4.6 Validation 8
5. Other Nonfunctional Requirements 9
5.1 Performance Requirements 9
5.2 Safety Requirements 9
5.3 Security Requirements 9
5.4 Software Quality Attributes 9
5.5 Business Rules 9

1. Introduction

1.1 Purpose

This Software Requirements Specification describes the functional and non-functional requirements for the CPU Scheduling Algorithm Simulator. This is a web application that is designed to help students visualize and understand CPU scheduling algorithms.

The current version, Version 1.0, will support the following cpu scheduling algorithms:

1. First-Come, First-Served
2. Shortest Job First
3. Shortest Remaining Time Firs
4. Round Robin
5. Priority Scheduling

1.2 Document Conventions

No special conventions.

1.3 Intended Audience and Reading Suggestions

This document is intended for me (The Developer), my project supervisor, and the professor for CISC 4900. To get a good idea of the scope of this project, read the first two sections. The rest of the sections are here to benefit me as a reference while I am building the project.

1.4 Product Scope

The CPU Scheduling Algorithm Simulator is an interactive web application where the user can input customized processes and is used to visualize how different CPU scheduling algorithms work. The program will generate a gantt chart, process table, average waiting/turnaround time, and throughput for each algorithm selected. This project benefits students and learners who need help verifying their gantt charts and sort out any confusion they might get when learning the algorithm for the first time. This project can be used by anyone who needs to make a visual representation for cpu scheduling algorithms and needs the flexibility of customization.

1.5 References

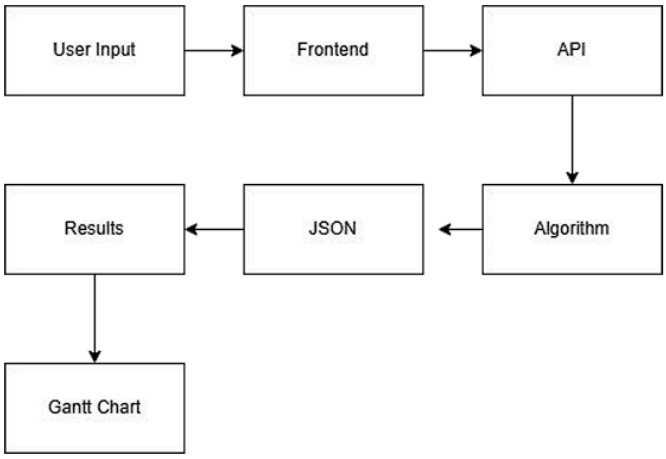
Documents and websites I used to help create this SRS document are:

1. srs-template-ieee.pdf
2. https://www.geeksforgeeks.org/software-engineering/software-requirement-specification-srs-format/

2. Overall Description

2.1 Product Perspective

This project is a web application that doesn't rely on any external systems, databases, or external APIs. We use a client-server architecture that has a frontend that collects the users input, sends the data to the backend through an API, the backend runs the algorithm, formats the results in JSON, sends it back to the frontend, and creates the gantt chart, process table, average waiting/turnaround time, and throughput.



2.2 Product Functions

1. Users enter the processes with attributes like arrival time, burst time, and priority.
2. Users select a scheduling algorithm.
3. The backend runs the scheduling algorithm with the processes and prepares the results.
4. A gantt chart is generated along with the process table, average waiting/turnaround time, and throughput.
5. Invalid inputs are handled and won't run unless all inputs are valid.

2.3 User Classes and Characteristics

The primary users are students who need help with CPU scheduling algorithms. These students may be enrolled in an Operating Systems course and need clear and visual feedback with a simple user interface. Another user is teachers who need a tool to demonstrate cpu scheduling algorithms to the class and need an accurate program that delivers correct results.

2.4 Operating Environment

This project will be supported on all operating systems. The only thing the user needs to access this project is a web browser and internet connection.

2.5 Design and Implementation Constraints

1. This system has to use Java and Spring Boot for backend development.

2. There must be no database connection because we are not storing any user data.
3. Round Robin is the only algorithm that requires quantum time to be selected.

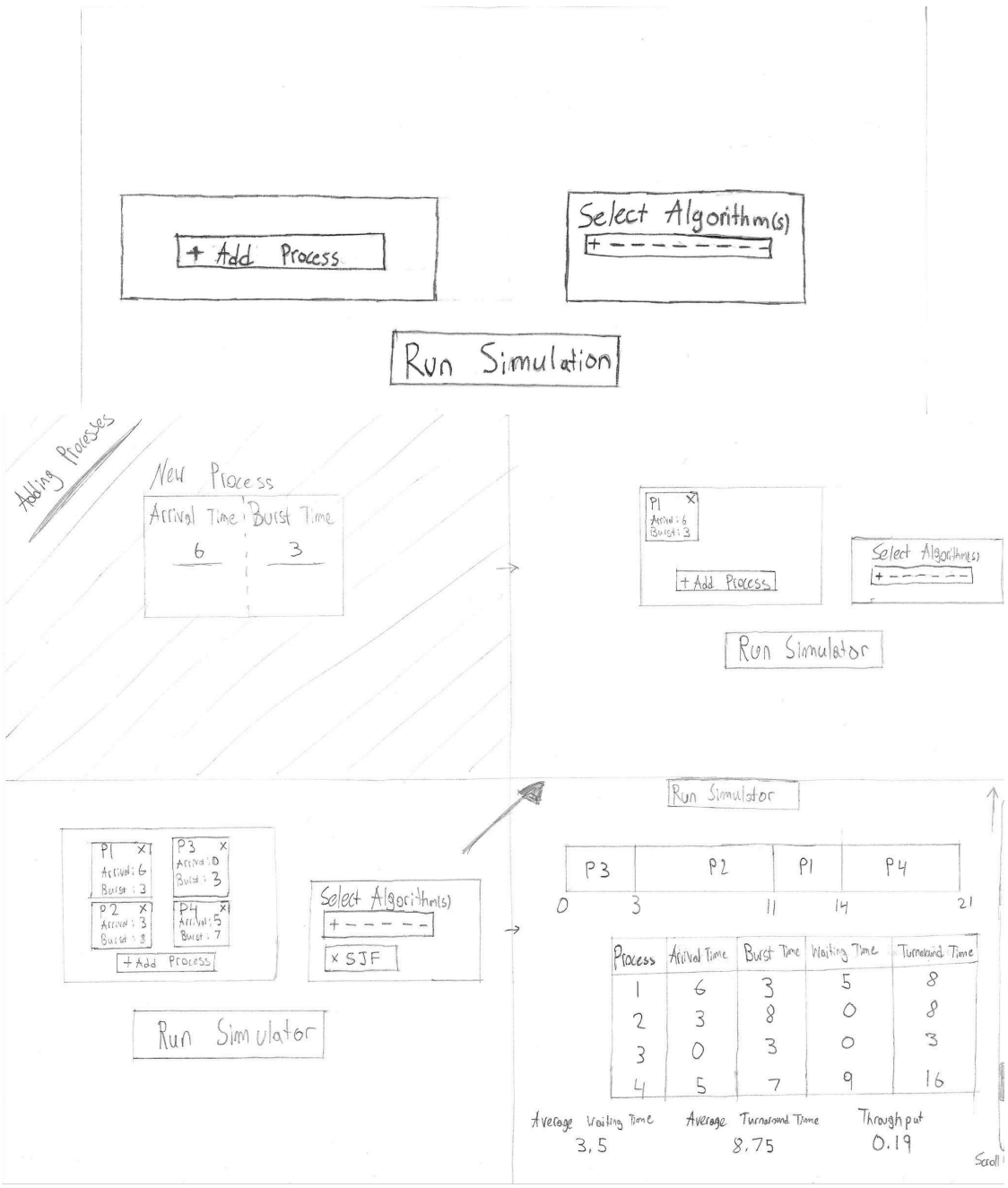
2.6 User Documentation

The user will have plenty of resources to learn how this project works. They will have access to:

1. A quick start guide for new users
2. Demo videos (The 2 Recordings)
3. Final presentation slides

3. External Interface Requirements

3.1 User Interfaces



3.2 Hardware Interface

Mobile and Desktop devices with access to the world wide web.

3.3 Software Interfaces

This project uses Spring Boot for the purpose of providing REST API endpoints, handling JSON, and managing dependency injection for cpu scheduling algorithms. All communication between the frontend and the backend will be from the use of JSON. There will be classes for the backend like Process, ScheduleRequest, ScheduleResult, GanttBlock, and ProcessResult. The external libraries that will be used are Lombok, Spring Starter Web, and Spring Starter Validation.

3.4 Communications Interfaces

This project uses HTTP calls for client-server communication and REST APIs for the backend endpoints. It will accept JSON requests and return a JSON request.
The simulator uses standard web communication protocols.

4. System Features

4.1 Input Processes

4.1.1 Description and Priority

This feature allows users to enter a list of processes with custom attributes that will be used in the simulation. It's a high priority feature because the simulator cannot run without any processes.
Priority Ratings from a scale of 1 - 9:

- Benefit: 9
- Penalty: 9
- Cost: 5
- Risk: 1

4.1.2 Stimulus/Response Sequence

1. User Action: User enters processes with arrival time, burst time, and priority.
   System Response: The system checks if the input is valid and updates the process list.

2. User Action: User tries to run the simulation with invalid data.
   System Response: The system displays an error message and doesn’t execute.

3. User Action: User adds or deletes a process from the process list.
   System Response: The user interface updates shows/removes the process.

4.1.3 Functional Requirements

REQ-1: The program must allow the user to input a list of processes.
REQ-2: Each process must include arrival time, burst time, and priority.
REQ-3: The program must validate that arrival time and burst time are positive numbers.
REQ-4: The program must not allow the user to run a simulation if the process list is empty.
REQ-5: The program must display error messages for invalid input.

4.2 Algorithm Selection

4.2.1 Description and Priority

This feature allows users to select what CPU scheduling algorithm they want to simulate. It's a high priority feature because without being able to select an algorithm, the program won't be able to run a simulation.
Priority Ratings:

- Benefit: 9
- Penalty: 9
- Cost: 3
- Risk: 1

4.2.2 Stimulus/Response Sequences

1. User Action: User selects any algorithm from the dropdown bar.
   System Response: If Round Robin is selected, the user interface requests the user to add a time quantum.

4.2.3 Functional Requirements

REQ-6: The program must allow users to select a scheduling algorithm.
REQ-7: The program must have the FCFS, SJF, SRTF, RR, and Priority Scheduling algorithms.
REQ-8: If Round Robin is selected, the program must ask for a quantum value.
REQ-9: The program must validate that the quantum is a positive number.
REQ-10: The program must now show the quantum option for all algorithms besides RR.

4.3 Simulation

4.3.1 Description and Priority

This feature runs the scheduling algorithm with all the processes as inputs. It's a high priority feature because it provides the results needed to make the Gantt chart, process table, average waiting/turnaround times, and throughput.
Priority Ratings:

- Benefit: 9
- Penalty: 9

- Cost: 7
- Risk: 1

4.3.2 Stimulus/Response Sequences

1. User Action: User runs the simulation.
   System Response: Validates the input, sends a request to the backend, backend runs the algorithm selected, backend returns results, frontend organizes results.

4.3.3 Functional Requirements

REQ-11: The program must validate all the inputs before running the simulation.
REQ-12: The program must send a JSON request that has the algorithm and processes.
REQ-13: The backend must run the scheduling algorithm that was chosen with the processes.
REQ-14: The backend must return the results to the frontend.

4.4 Gantt Chart Visualization

4.4.1 Description and Priority

This feature displays the cpu scheduling algorithm gantt chart to the user. It's a high priority feature because this is the whole reason the user would want to use this project for. To visualize the cpu scheduling algorithm of their choice.
Priority Ratings:

- Benefit: 9
- Penalty: 9
- Cost: 6
- Risk: 1

4.4.2 Stimulus/Response Sequences

1. User Action: User waits after running the simulation.
   System Action: When the backend returns the results, the frontend organizes the data and creates a nice visual for the user to understand.

4.4.3 Functional Requirements

REQ-15: The program must use the backend results to create a gantt chart.
REQ-16: Each gantt block must show the process with the start and end times.
REQ-17: The gantt chart should be easy to look at with different colors for each process.

4.5 Process Table and Relevant Data

4.5.1 Description and Priority

This feature displays the processes table after execution along with the average waiting/turnaround time and throughput. It's a high priority feature because users rely on this data to verify their work relating to cpu scheduling algorithms.
Priority Ratings:

- Benefit: 9
- Penalty: 9
- Cost: 4
- Risk: 1

4.5.2 Stimulus/Response Sequences

1. System Action: After the simulation, the program displays the process table along with the average waiting time, turnaround time, and throughput.
   User Action: User reviews the results and compares their solution or verifies their homework.

4.5.3 Functional Requirements

REQ-18: The program must display the waiting and turnaround times for each process.
REQ-19: The program must calculate and display the average waiting and turnaround times.
REQ-20: The program must calculate the throughput.
REQ-21: The program should display the results in a table.

4.6 Validation

4.6.1 Description and Priority

This feature makes sure that the system runs as expected by making sure the data the user entered is valid. It’s a high priority because making sure that the algorithm functions properly when run is important for my project's reputation. I don’t want to have a project that claims to simulate something but ends up doing it incorrectly.
Priority Ratings:

- Benefit: 9
- Penalty: 9
- Cost: 4
- Risk: 1

4.6.2 Stimulus/Response Sequences

1. User Action: User enters invalid input.
   System Response: Programs displays an error message and doesn't start the simulation.

2. User Action: User enters correct inputs thanks to the error message.
   System Response: Error message disappears and the simulation can be run.

4.6.3 Functional Requirements

REQ-22: The program must be able to detect invalid inputs.
REQ-23: The program must prevent the user from starting a simulation when their inputs are invalid.
REQ-24: The program should provide clear error messages explaining the problem.
REQ-25: The program should visually show the field is invalid using colors like red.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The CPU Scheduling Simulator is an educational tool that has to return results quickly.

5.2 Safety Requirements

N/A

5.3 Security Requirements

This project does not collect user data so there is no chance of a data leak from attackers but the project should be deployed on HTTPS instead of HTTP to add more security for the user.

5.4 Software Quality Attributes

The user interface should be simple for first time users to navigate and understand how it works without any tutorials. The user only has three steps they need to complete to run the simulation. They have to create the processes, select the algorithm, and press run to start the simulation. All the CPU scheduling algorithms will be implemented correctly and won't be published if it doesn't work as expected.

5.5 Business Rules

1. The expected users are students, instructors, and learners of operating systems.
2. The program has to be accessible without logging in and collecting user data.
3. The program cannot track the users.
4. The simulator has to support FCFS, SJF, SRTF, RR, and Priority scheduling algorithms.